Module 4 Assessment Project

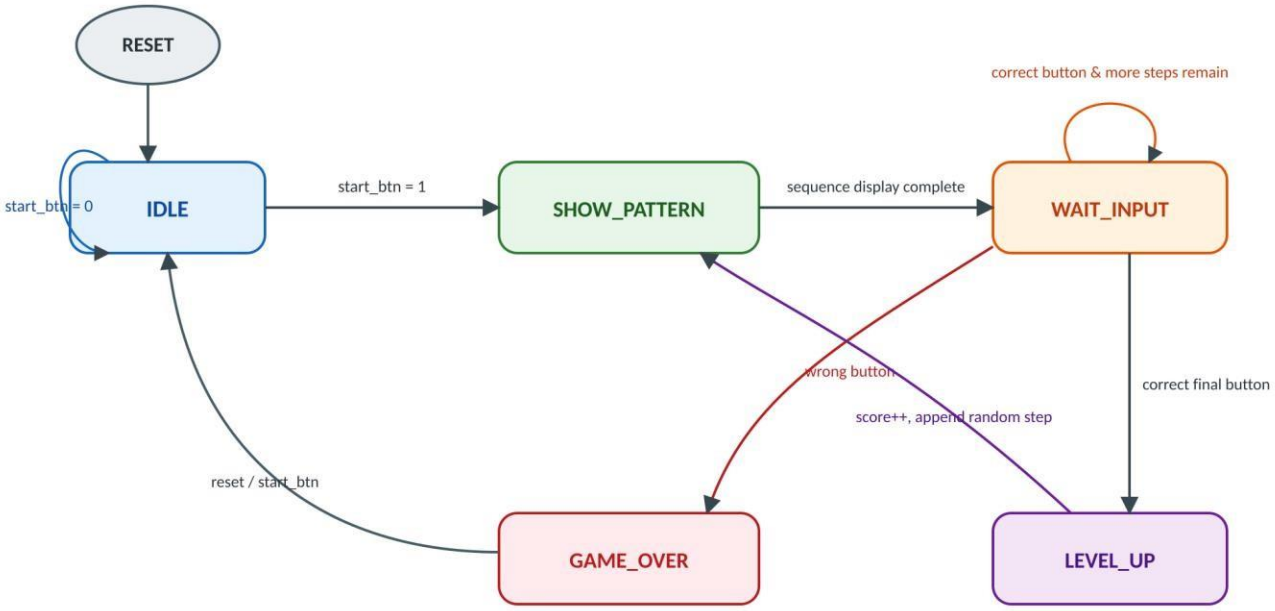
Design Phase of Simon Says Memory Game

1. FSM Diagram (States, Transitions, and Conditions)

1.1 State Descriptions

State	Description
IDLE	Initial state after reset. Waits for the Start button. Score is cleared and outputs remain inactive.
SHOW_PATTERN	Displays the stored LED sequence one step at a time using the clock divider.
WAIT_INPUT	Waits for the player to reproduce the displayed sequence. Each button press is compared with the stored sequence.
LEVEL_UP	Executed when the player correctly enters the complete sequence. Score is incremented and one new random step is appended.
GAME_OVER	Entered immediately after an incorrect button press. Score is frozen and the system waits for reset or restart.

1.2 FSM State Transition Diagram

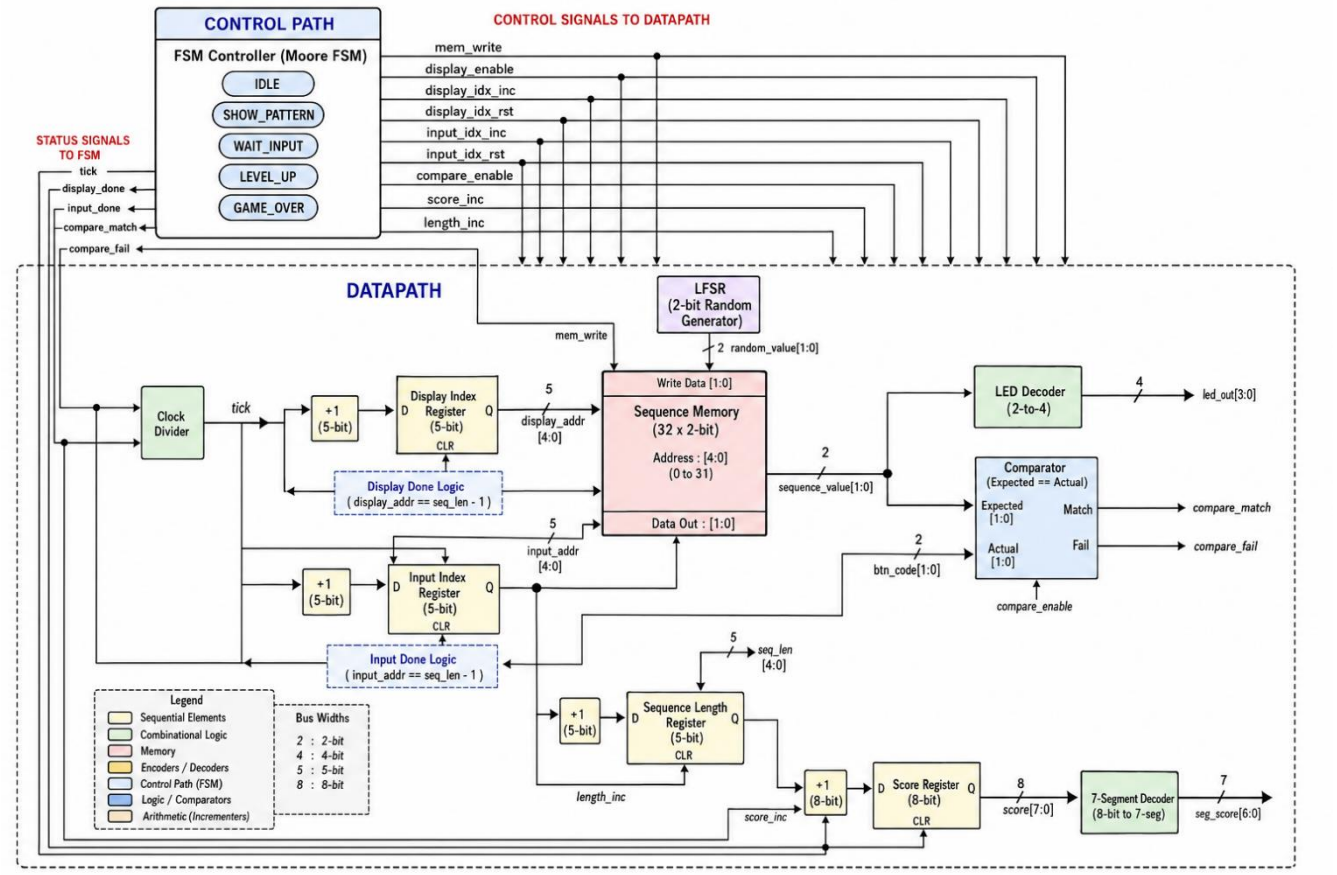


1.3 State Transition Table

Current State	Condition	Next State
IDLE	start_btn = 0	IDLE
IDLE	start_btn = 1	SHOW_PATTERN
SHOW_PATTERN	Sequence display complete	WAIT_INPUT
WAIT_INPUT	Correct button & more steps remain	WAIT_INPUT
WAIT_INPUT	Correct final button	LEVEL_UP
WAIT_INPUT	Wrong button	GAME_OVER
LEVEL_UP	New step appended	SHOW_PATTERN
GAME_OVER	reset / start_btn	IDLE

2. Block Diagram

The block diagram below shows the module boundaries and the connection between the Control Path (FSM) and the Datapath.



2.1 Module Responsibilities

Module	Function
FSM Controller	Controls the complete game operation.
Sequence Memory	Stores the generated LED sequence.
LFSR Generator	Produces a new pseudo-random sequence element.
Clock Divider	Generates a slow clock for human-visible LED timing.
Display Index Counter	Tracks which LED in the sequence is currently being shown.
Input Index Counter	Tracks which player input is expected next.
Comparator	Compares player input with the stored sequence value.
Score Counter	Counts completed rounds.
Seven Segment Driver	Converts score into a seven-segment display pattern.

3. Moore vs. Mealy Choice

Selected FSM Type: Moore FSM

Justification

The Simon Says Memory Game is implemented using a Moore Finite State Machine because the primary outputs depend on the current state rather than directly on the current inputs. For example:

- In the SHOW_PATTERN state, the LEDs display the stored sequence.
- In the GAME_OVER state, the game_over output remains asserted.
- During IDLE, the system waits for the start button with no active outputs.

The player button inputs are used only to determine state transitions. For example, a correct button press causes a transition from WAIT_INPUT to either WAIT_INPUT or LEVEL_UP, while an incorrect button press causes a transition to GAME_OVER. The outputs change only after the FSM enters the next state.

A Moore implementation was selected because it provides:

- Simpler and more organized state-based output logic.
- Stable outputs that change only on state transitions.
- Easier debugging and verification.
- Better alignment with the state-oriented behavior described in the project specification.